

SAVANNA

SENTINEL



Software Architecture Specification

Last Revision: 2 September 2026

David van Rooijen

Stephan Kritzinger

Dandré Nel

Micheal Koch

Daniel Cohen

TABLE OF CONTENTS

Introduction.....	2
Architectural Requirements.....	2
Factory.....	3
Memento.....	3
Observer.....	3
Technology Requirements.....	9
Service Contracts.....	21
Deployment.....	22
Production.....	23
Development.....	24
NFR Traceability Matrix.....	26

Introduction

This document indicates the architectural decisions that were made along the project's lifespan, and includes justification for these decisions. This includes the scaffolding for the project to run, the technology decided to implement the architectural plan, the API endpoint documentation, and how the application is deployed.

Architectural Requirements

Architectural Patterns

We use the N-Layered(5-Layered) architecture approach

- **Layer 1: Presentation Layer:** Contains everything that the user is shown, and receives responses from the bottom layers to determine what it should review
- **Layer 2: Reverse Proxy:** Any requests sent from the frontend will pass through the reverse proxy, which will ensure the request is encrypted.
- **Layer 3: Controller Layer:** Parses HTTP requests, validates request shape, enforces role-based access control via token inspection, and returns response models. Contains no business logic.
- **Layer 4:**
 - **Email Layer:** Handles sending emails to users when required.
 - **Async Processing Layer:** Contains the business logic of the program.
- **Layer 5: Data Layer:** Executes all database queries via an async session. Contains no business logic.

Design Patterns

Factory

A factory method is used in the backend to perform role based authentication. There exists functionality to specify which roles are able to access an endpoint into the factory, and the factory shall return a function that validates these roles against the current user's role, removing the need to have a separate function for each role combination that can access an endpoint.

Memento

Memento will be used when the timeslider is implemented, allowing the state of the heatmap to change to previous versions in time easily.

Observer

Observer is used for the notification system. The backend will queue a notification, which the frontend will observe to determine if it should display the notification or not.

Constraints

Constraint	Description
Open Source Requirement	All components and third-party libraries must be open-source. This avoids licensing costs, vendor lock-in, and ensures the solution remains maintainable beyond the project lifecycle.
Deployment Requirement	The system must be fully containerised using Docker. All services must be orchestrated through Docker Compose and deployable via a single docker compose up command with no manual configuration steps. GitHub Actions is used for CI/CD to validate builds and run tests before any merge to the main branch. The production environment must enforce HTTPS and role-based access control.
Progressive Web App Requirement	The system must be delivered as a PWA. This is a hard requirement driven by the field environment where rangers operate without guaranteed connectivity. The PWA must support offline data capture and synchronise with the central database once connection is restored.
Data Sensitivity	The system handles sensitive conservation data including poaching incident locations, patrol routes, and informant tip-offs. All data in transit must be encrypted via HTTPS, and sensitive map layers must be restricted by authenticated user roles.

Budget Constraint	The total project budget provided by EPI-USE is capped at R5000. All cloud services, third-party integrations, and tooling must be operated within this limit.
-------------------	--

Architectural Diagram

Original Image can be found at [docs > demo2 > architecture](#), if a higher resolution is required

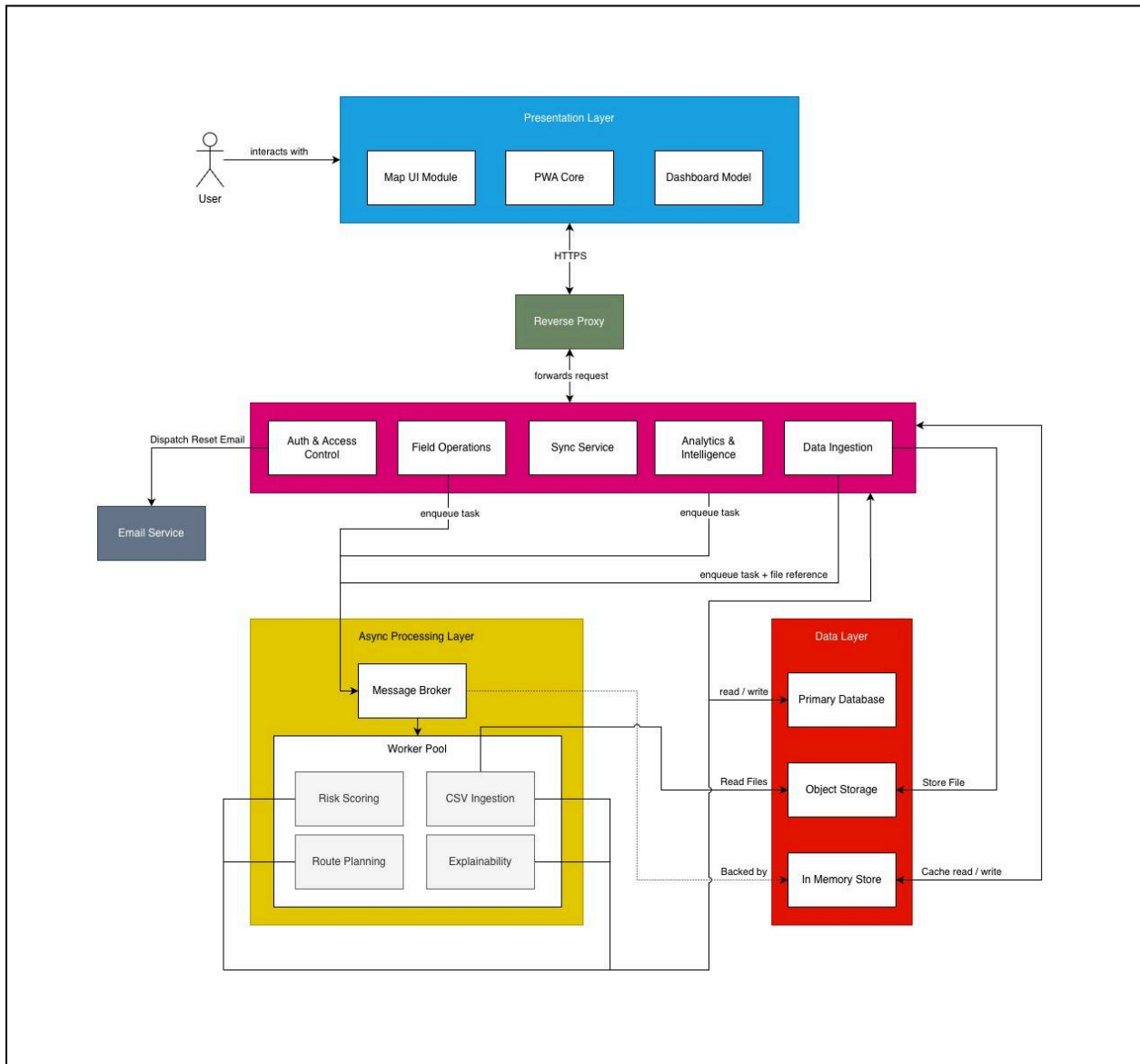


Figure 1 - Architectural Diagram

Quality Requirement Mapping

Quality Requirement	Architectural Decision
Security	Implementation of HS256/TLS1.3 encryption, and the use of JWT with RBAC to enforce API endpoint access.
Reliability $\geq 99.9\%$ uptime	Docker Compose restart policies ensure automatic recovery of crashed containers. Additionally the PWA functionality ensures that the application is still able to function even if the server becomes temporarily unavailable.
Performance for 60 concurrent users	FastAPI's async I/O prevents blocking during concurrent requests. PostGIS GiST indexing and Redis caching will also improve performance during heavy queries, while the client will use MapLibre GL to handle client-side rendering, improving performance.
Maximum of 2 hours downtime in a month	CI/CD prevents broken code from reaching our production, and in the case of downtime, our redeploy previous commit strategy ensures a rapid recovery within 2 hours if deployment fails.

Maintainability	• Implementation of Github actions using its CI pipeline, running an automated test suite to ensure that requirements are matched. • Implementation of SonarQube to point out maintainability issues in code, to ensure easy modification of code in the future. • Codecov ensures that test coverage hits at least 75%
-----------------	---

Technology Requirements

The technology requirements for Savanna Sentinel encompass the architectural and delivery requirements that ensure the system supports a reliable, scalable, performant, and secure Progressive Web App (PWA). We have taken into consideration the following technology requirements while choosing the tech stack for Savanna Sentinel.

The technology requirements for Savanna Sentinel encompass the architectural and delivery requirements that ensure the system supports a reliable, scalable, performant, and secure Progressive Web App (PWA). We have taken into consideration the following technology requirements while choosing the tech stack for Savanna Sentinel.

Technology Stack Overview

Purpose	Solution
Frontend	React 19 (Typescript)
UI Components	shadcn/ui + Tailwind CSS v4
Mapping	MapLibre GL
PWA / Offline	Workbox + Dexie.js (IndexedDB)
Backend API	Python FastAPI
Background Jobs	Celery + Redis
Database	PostgreSQL + PostGIS
Object Storage	Seaweed
AI / ML	sckit-learn + GeoPandas + pandas + SHAP
Security	JWT(PyJWT, HS256)
Reverse Proxy	Caddy
Email	Resend SDK
DevOps / CI	Docker + Github Actions

Frontend

Chosen Technology: React with Typescript

React was explicitly recommended by our client EPI-USE in the project proposal. Beyond the client's preference, it is the most suitable choice for the following reasons:

- **PWA support:** React integrates natively with Workbox service workers, satisfying the offline field capture requirement without an additional framework layer.
- **Type Safety:** TypeScript catches type mismatches between geospatial API responses and map layer data at compile time, reducing the class of bugs most likely to occur.
- **MapLibre GL integration:** The `react-map-gl` binding for MapLibre GL supports the heatmap layer, time range slider, and role based layer visibility without custom WebGL code.'
- **Component isolation:** The explainability panel, patrol comparison interface, and dashboard cards can be developed as independent components, supporting parallel development.

Alternative considered - Angular: Angular was listed in the proposal as an option. We decided against it, because the team's existing React/TypeScript proficiency would have required relearning Angular's dependency injection and zone based change detection, reducing development velocity without any functional benefit for this project.

UI Components & Styling

Chosen Technology: shadcn/ui with Tailwind CSS

shadcn/ui is a React component library built on Radix UI primitives and styled with Tailwind CSS utility classes.

- **Ownership model:** shadcn components are copied directly into the project codebase rather than installed as a black-box dependency. This means every component (buttons, forms, modals, navigation) can be customised to match the Savanna Sentinel brand palette and typography without fighting an external design system.
- **Tailwind Integration:** Tailwind's utility-first CSS classes provide precise layout and spacing control across the dashboard, map interface, and mobile PWA forms without writing custom CSS files.
- **Lucide Icons:** shadcn ships with Lucide as its default icon set, which is already specified in the Brand Style guide. No additional icon library is required.
- **Accessibility:** shadcn components are built on Radix UI, which provides ARIA roles, keyboard navigation, and focus management out of the box, directly supporting the WCAG 2.1 AA accessibility requirements.
- **Radix UI primitives:** Complex interactive components such as modals, dropdowns, sliders (time range), and tooltips (explainability panel) are handled by Radix UI's headless primitives, ensuring correct behaviour without custom implementation.

Alternative considered - Bootstrap: Bootstrap was considered due to team familiarity. It was not selected because it ships its own opinionated CSS that conflicts with Tailwind utility classes, requires a separate react-bootstrap wrapper package to function properly in React, and makes it significantly harder to apply a custom brand palette without overriding its defaults. shadcn + Tailwind provides the same component coverage with full control over styling and no class-name conflicts.

Mapping

Chosen Technology: MapLibre GL with OpenStreetMap tiles

MapLibre GL renders map tiles and overlay layers using the device GPU via WebGL rather than browser side JavaScript loops, which is critical for the project

- **Heatmap performance:** Risk heatmaps covering large reserves (thousands of grid cells) must render smoothly as the user drags the time slider. CPU based libraries such as Leaflet with canvas heatmap plugins stutter above approximately 5 000 points on mid range devices.
- **No licensing cost:** MapLibre GL is fully open source using free OpenStreetMap tiles.
- **Vector tile support:** MapLibre supports vector tile sources, enabling role based layer toggling at the client without additional server round trips. Sensitive layers can be managed by role based access control.

Alternative considered - Leaflet: Leaflet was evaluated due to its simplicity, large plugin ecosystem, and ease of integration with React. However, it was not selected because it relies primarily on CPU based rendering, which does not scale efficiently for large geospatial datasets. In performance testing scenarios relevant to this project (thousands of spatial points and dynamic heatmaps), Leaflet exhibits noticeable lag and reduced frame rates on mid range devices. Additionally, Leaflet lacks native vector tile support and GPU acceleration, requiring additional plugins and workarounds that increase complexity without achieving MapLibre GL's performance characteristics

PWA / Offline

Chosen Technology: Workbox (service workers) with Dexie.js as a typed IndexedDB wrapper for structured offline storage.

Offline capability is a first class requirement. Rangers operate in reserves with no mobile data coverage.

- Workbox precaches the application shell so the app loads without a network connection. The Background Sync API queues POST requests (incident reports, sightings) and replays them automatically when connectivity is restored.
- Dexie.js provides a clean, typed API over IndexedDB to store draft field reports locally on the device. Timestamp based conflict resolution is applied server side when the upload reaches the FastAPI backend.
- Photo uploads are queued as blob references in IndexedDB and uploaded to Seaweed once online, avoiding data loss for large files.

Backend API

Chosen Technology: Python FastAPI

FastAPI was recommended by the client and is the optimal choice for reasons beyond the client's preference.

- **OpenAPI auto generation:** FastAPI produces an /openapi.json specification automatically from Python type annotations, satisfying the architectural requirement for a versioned REST API with OpenAPI documentation without additional tooling.
- **Async I/O:** FastAPI runs on ASGI, enabling concurrent request handling for multiple rangers submitting field reports simultaneously without blocking.
- **Python ecosystem co-location:** The AI/ML modules (scikit-learn, GeoPandas, pandas, SHAP) run in the same Python environment as the API, eliminating the inter process serialisation overhead that would exist if the ML layer were written in a different language.
- **RBAC middleware:** FastAPI's dependency injection system is well suited to implementing role-checking middleware without cross cutting boilerplate.

Alternative considered - Node.js: Node.js was listed in the proposal. It was not selected because scikit-learn and GeoPandas do not have production equivalent JavaScript alternatives. Using Node.js would have required a separate Python microservice for ML, adding an inter service boundary and increasing operational complexity.

Background Processing

Chosen technology: Celery with Redis as the message broker

Risk score computation and heatmap generation are computationally expensive operations that must not block synchronous API requests.

- Redis as the message broker is lightweight, easy to containerise, and provides the pub/sub mechanism Celery requires. The same Redis instance is reused for API level response caching, reducing repeated PostGIS query load.
- Celery Beat allows periodic model retraining and heatmap refresh to be configured as cron style tasks, satisfying the architectural requirement for background jobs without manual intervention.
- Worker isolation ensures a long running scoring job cannot starve API workers, preserving the system's performance quality attribute.

Database

Chosen technology: PostgreSQL with PostGIS

The EPI-USE architectural requirements explicitly mandate PostGIS spatial indexing.

Spatial indexing: PostGIS GiST and BRIN indexes support the spatial operations the risk engine depends on, `ST\_Within`, `ST\_DWithin`, and `ST\_MakeGrid`, at the scale of millions of historical incident records.

ACID compliance: Field report data is safety critical. PostgreSQL transactional guarantees ensure a partial sync from a ranger's device either fully commits or rolls back, preventing corrupt sighting records.

JSONB columns: Heterogeneous tip off payloads and field report attachments are stored in JSONB columns, providing flexible schema within typed columns without sacrificing queryability.

Row-Level Security (RLS): PostgreSQL RLS policies enforce role based data redaction at the database layer as a defence-in-depth measure supplementing application level RBAC.

Object Storage

Chosen technology: Seaweed

CSV uploads and optional photo classifications require blob storage separate from the relational database.

- **API compatibility:** The application uses the standard boto3 Python SDK. If the project later migrates to AWS S3 or another provider, no code changes are required - only an environment variable update.
- **Self hosted within budget:** Seaweed runs as a Docker container within the same Compose stack. There are no per-GB transfer fees during development.
- **Pre-signed URLs:** Allow the frontend to upload photos directly to Seaweed without routing large files through the FastAPI backend, reducing server load.

AI / ML

Chosen technology: scikit-learn, GeoPandas, pandas, SHAP

The AI Risk Engine requires risk scoring, spatial feature engineering, explainability metrics, and general data processing. This combination maps directly to those four concerns.

scikit-learn: Provides gradient boosting and ensemble classifiers (Random Forest, XGBoost-compatible pipelines) that are appropriate for tabular spatio temporal incident data. These models outperform deep learning on small structured datasets typical of reserve scale historical records.

GeoPandas: Handles spatial feature engineering: converting incident coordinates into grid cells, computing kernel density estimates per cell, and joining patrol coverage data with incident locations.

pandas: Used alongside GeoPandas for general data manipulation, cleaning, transformation, and preprocessing of tabular datasets before spatial operations and model training.

SHAP (SHapley Additive exPlanations): Generates per-cell explainability metrics, detailing the contribution of each feature to a given risk score. This satisfies the functional requirement for an explainability panel that communicates model reasoning without overclaiming certainty.

Security

Chosen technology: PyJWT with HS256 symmetric signing

Authentication and RBAC require a stateless session mechanism compatible with both the web dashboard and PWA offline mode.

- **Stateless tokens:** JWT tokens are self contained, enabling FastAPI to validate requests without a database round trip per request, which is important for high frequency map tile and sync requests.
- **HS256 symmetric signing:** A single shared secret is used to both sign and verify tokens. This is appropriate for a monolithic deployment where all services share the same secret via environment variable.
- **Role claims in token payload:** The user's role (Ranger, Analyst, Community Liaison, Admin) is encoded in the JWT. FastAPI dependency injection reads this claim to enforce view and data restrictions without querying the database on each request.
- **Refresh token tracking:** Issued refresh tokens are stored in the database with their JTI, expiry, and revocation timestamp, enabling secure logout and token invalidation without sacrificing stateless access token validation.
- **HTTPS enforcement:** All endpoints are served over TLS via a Caddy reverse proxy with automatic certificate provisioning.

Reverse Proxy

Chosen technology: Caddy

All external traffic passes through Caddy before reaching the FastAPI backend.

- **Automatic TLS:** Caddy provisions and renews TLS certificates automatically, satisfying the requirement that no endpoint is accessible over unencrypted HTTP without manual certificate management.
- **Simple configuration:** Caddy's declarative Caddyfile syntax reduces operational overhead compared to alternatives such as Nginx, with no separate certbot process required.
- **Containerised deployment:** Caddy runs as a Docker container within the same Compose stack, keeping the full system deployable via a single docker compose up command.

DevOps / CI

Chosen technology: Docker with Docker Compose and GitHub Actions

The architectural requirements require containerised deployment and a CI pipeline with automated testing.

- **Docker Compose:** Defines the full local environment (FastAPI, Celery, Redis, PostgreSQL+PostGIS, Seaweed, Caddy) as a single `docker compose up` command, ensuring all team members develop against identical dependencies.
- **GitHub Actions:** Runs on every push and pull request to main. The pipeline executes backend unit and integration tests (pytest), frontend unit tests (Vitest), SonarCloud static analysis, and Coveralls coverage upload. Merging to main is blocked if any stage fails.
- **Branch protection rules:** Enforce mandatory pull requests and passing CI checks before merging.
- **The main branch:** Always reflects a deployable state.

Transactional Email

Chosen technology: Resend (SMTP relay) with the resend Python SDK

A transactional email service is required to deliver password reset links generated by the password recovery flow.

- Resend provides a developer-focused SMTP relay with a free tier (100 emails/day) that operates within the project budget constraint.
- **SDK integration:** The resend Python package integrates directly into the FastAPI service layer. No additional worker or container is required; reset emails are dispatched inline during the forgot-password request handler.
- **Open standard:** Resend uses standard SMTP and a minimal REST SDK. Migrating to an alternative provider (Mailgun, SendGrid, AWS SES) requires only an environment variable and SDK swap with no architectural change.
- **No self-hosted mail server:** Running a self-hosted SMTP server (Postfix, Mailhog in production) within the Docker Compose stack was considered but rejected due to deliverability risks and the operational overhead of managing SPF/DKIM records within the project timeline.

Service Contracts

Visible at savannasentinel.co.za/v1/openapi.json or savannasentinel.co.za/v1/docs

Deployment

More details of deployment can be found in the dedicated deployment doc at [docs > demo2 > deployment or cicd](#)

Deployment Area

<https://savannasentinel.co.za/>

Environmental Parity: main deploys to the website automatically, and we have a development environment in the github.

Containerisation: Each layer and application is packaged as a seperate docker container, it is not microservices, but various services are in seperate containers when convenient

List of containers:

- Frontend
- Backend
- DB
- Redis
- Seaweed

Secrets Management: Github secrets are used for the development environment, such as for e2e testing, while a .env file is securely stored on the AWS

Rollback Strategy: Redeploy previous commit is the strategy that is used for rollbacks.

Deployment Diagram

Production

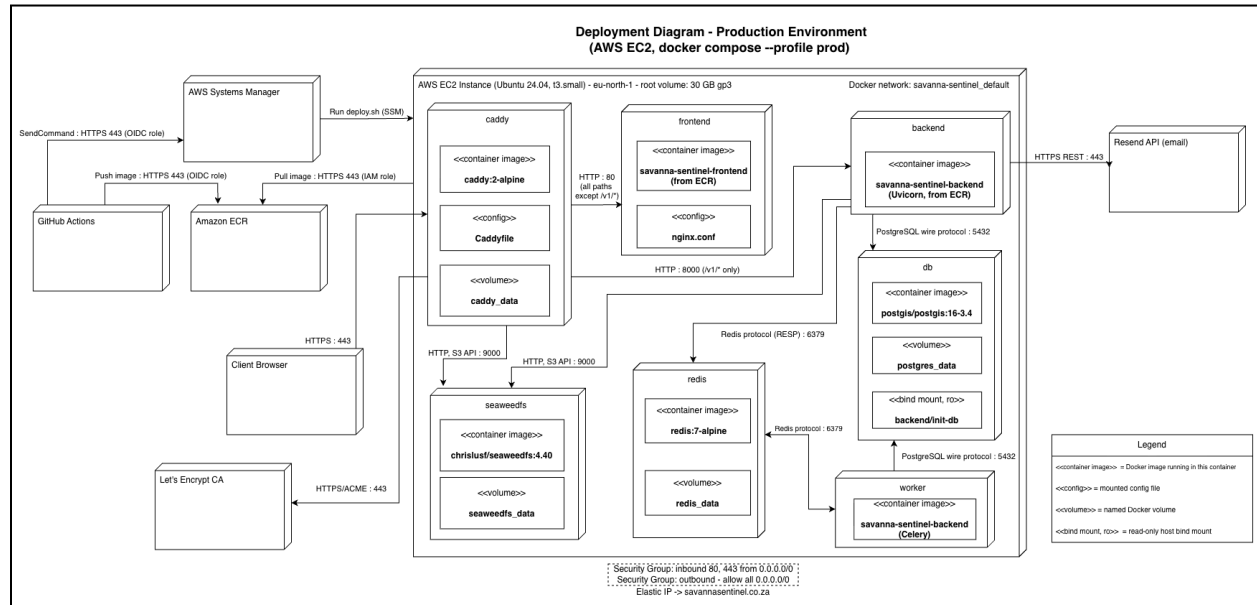


Figure 2 - Production Deployment Graph

Development

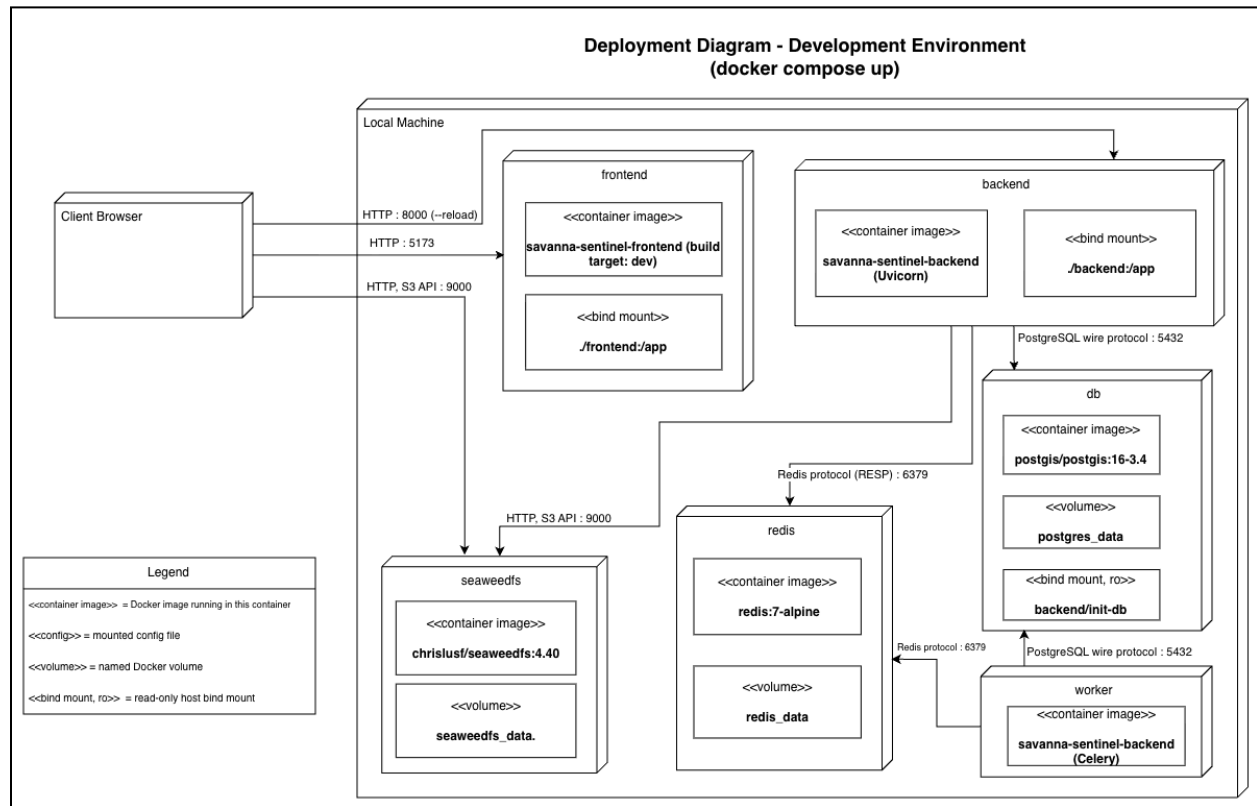


Figure 3 - Development Deployment Graph

CI/CD

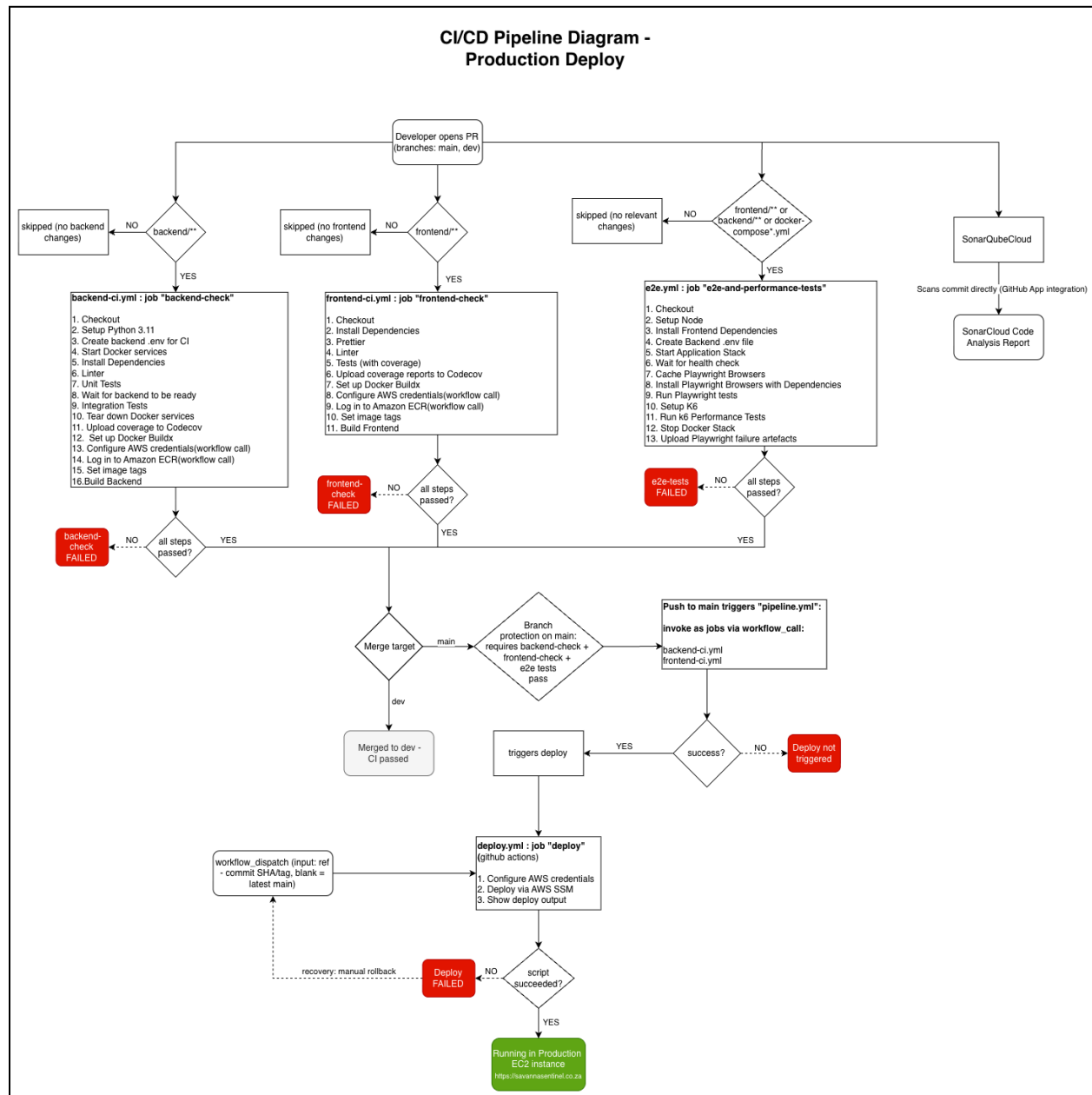


Figure 4 - CI/CD Workflow

NFR Traceability Matrix

ID	Quantified Requirement	Tactic in SAS	Test / Tool	Target / Actual
NFR-1	Sensitive user data encrypted (AES-256/TLS1.3), RBAC enforced on API endpoints	AES-256/TLS1.3 encryption and the use of JWT with RBAC to enforce API endpoint access	Mozilla HTTP Observatory Badge, Bandit, pip-audit, npm audit	Observatory: A+/A+ Bandit: 0 High (Severity) / 0 High Severity pip-audit: 0/0 npm-audit: 0/0
NFR-2	>= 99.9% uptime	Docker Compose restart policies ensure automatic recovery of crashed containers, PWA functionality ensures the application still works even if the server becomes temporarily unavailable	UptimeRobot Badge	>= 99.9% / 96.4% (Read NFR-4 for reasons)
NFR-3	Respond within 2s for 95% of requests, support >= 60 concurrent users	FastAPI's async I/O prevents blocking during concurrent requests, PostGIS GiST indexing and Redis caching improve performance during heavy queries, MapLibre GL handles client-side rendering	k6 performance tests, approximately 60 concurrent users, running on every merge to ensure performance	p95 <= 2s / p95 <= 1.4s

NFR-4	<= 2 hours downtime per month	CI/CD prevents broken code from reaching production, redploy previous-commit strategy ensures rapid recovery within 2 hours if deployment fails	Automated container restarts, UptimeRobot status badge	<= 2h per month / ~24h due to github actions outage leading to a failing deployment. Actions have been taken to remedy this for the future
NFR-5	New features/bug fixes deployable within 2 hours	Same CI/CD + redploy tactic as NFR-5	GitHub Actions CI must pass (regression gate) and measured pipeline duration (commit -> CI pass -> deploy complete) via Actions run history	All tests pass and <=2h wall-clock / Max action time is ~10m
NFR-6	>= 75% automated test coverage	GitHub Actions run an automated test suite, SonarQube points out maintainability issues, Codecov ensures test coverage hits at least 75%	Unit and Integration tests, End-to-End tests, Backend CI Badge, Frontend CI Badge, Codecov Badge	>= 75% / 91%